



ELSEVIER

Contents lists available at ScienceDirect

Future Generation Computer Systems

journal homepage: www.elsevier.com/locate/fgcs

A decentralized asynchronous federated learning framework for edge devices

Bin Wang^a, Zhao Tian^{b,d}, Jie Ma^b, Wenju Zhang^b, Wei She^{b,d,e}, Wei Liu^{b,c,d,*}^a School of Computer and Artificial Intelligence, Zhengzhou University, Zhengzhou 450000, China^b School of Cyber Science and Engineering, Zhengzhou University, Zhengzhou 450000, China^c Henan Key Laboratory of Network Cryptography Technology, Zhengzhou, 450000, China^d Zhengzhou Key Laboratory of Blockchain and Data Intelligence, Zhengzhou, 450000, China^e SongShan Laboratory, Zhengzhou 450000, China

ARTICLE INFO

Keywords:

Federated learning

Blockchain

Smart contract

Edge device

ABSTRACT

The traditional synchronous federated learning framework ensures global model consistency and accuracy. However, it is limited by the computational power differences between devices and the influence of non-IID data, which leads to inefficient training and insufficient model generalization performance. In this paper, we propose a decentralized asynchronous federated learning framework. The framework uses smart contracts deployed on the blockchain to manage edge devices for enhanced flexibility. At first, the framework performs model aggregation and validation through the use of consensus groups. It eliminates the potential single point of failure associated with centralized parameter servers. In addition, we propose a Federated Learning with Dynamically Growing Cache (FedDgc) method in a non-IID environment. The method reduces redundant gradient information exchange during initial feature extraction while maintaining the learning capability of the global model. Finally, the experimental results show that our framework has better test performance and guarantees the convergence speed of the model during training.

1. Introduction

With the advent of the big data era, the surge in data volumes and network devices brings with it serious security risks. The independent definition and storage of data between different organizations leads to the problem of data silos. Traditional distributed learning struggles to overcome these data barriers [1–3]. Google proposes a federated learning (FL) framework to address this issue, which seeks to accomplish multi-party collaborative training while simultaneously protecting data privacy [4]. To address the issues of data silos and privacy, FL specifically uses a model-parallel approach that keeps the raw data local and only transfers model parameters between devices [5–7].

A typical FL framework consists of a central server and multiple edge devices participating in training. However, the edge devices exhibit heterogeneity due to differences in factors such as network conditions, energy resources, and computational power [8,9]. Moreover, the global update speed is affected by the slowest participating device during the training process, which is called the “straggler problem” [10]. To solve the above problem, many researchers have proposed client selection mechanisms and asynchronous FL methods. The client selection mechanism selects training participants based on the state of the edge devices [11–13]. Although it can reduce the waiting time of the parameter server, it will exclude edge devices that have representative

clients. Among them, Dong et al. propose a client selection strategy based on the amount of information (AoI). This scheme ensures fairness by allowing all clients to submit model updates. It also uses a two-step sub-optimal algorithm to compute client priorities and then selects the highest priority client for updating [14]. The communication resource constrained hierarchical federated learning (CRCHFL) framework proposed by Kou [15]. It optimizes communication resources through hybrid data and model fusion to minimize the generalization error of the autopilot model. These approaches have achieved significant improvements in ensuring fairness and optimizing resources. However, the client selection mechanism lacks the flexibility to cope with complex and changing real-world scenarios. In contrast, asynchronous FL allows each client to train and submit updates independently of other devices without waiting for the completion of other clients [16,17]. While this approach improves edge device participation and feedback speed, it causes model inconsistency when targeting large amounts of Non-IID data [9].

Although the above methods improve the system efficiency to some extent. But in practice, the FL framework is responsible for the aggregation and updating of all models through a central server [18]. When it fails, the training results of all edge devices cannot be aggregated. This will lead to the interruption of the whole federated

* Corresponding author at: School of Cyber Science and Engineering, Zhengzhou University, Zhengzhou 450000, China.

E-mail address: wliu@zzu.edu.cn (W. Liu).

<https://doi.org/10.1016/j.future.2024.107683>

Received 15 July 2024; Received in revised form 7 December 2024; Accepted 14 December 2024

Available online 31 December 2024

0167-739X/© 2024 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

learning process, which is called the single-point-of-failure problem. To address this challenge, blockchain technology has emerged as a decentralized solution [19]. First, the decentralization, invariance, and traceability characteristics of blockchain are highly compatible with the FL framework. By building a blockchain consensus group, it can effectively prevent the risk of tampering with the central server and free itself from the dependence on the centralized server [20]. In addition, the smart contract on the blockchain provides a flexible interface for the FL framework, which can efficiently manage the training process of edge devices [21]. The security and efficiency of the system are further enhanced.

In summary, this paper proposes a decentralized asynchronous federated learning framework for edge devices. The method utilizes a consensus group instead of a centralized server for model aggregation. Within this framework, we propose a dynamic growth cache federated learning (FedDgc) method under Non-IID settings. During the global model iteration process, FedDgc uses a caching mechanism to receive local models from edge devices, aggregating the model when the cache reaches the update threshold. This method alleviates the effect of local models on the global model. Furthermore, it adjusts cache capacity dynamically based on training progress to decrease the exchange of redundant gradients. Moreover, during the model aggregation, FedDgc incorporates staleness weights and dataset volume weights to enhance the accuracy and fairness of the global model. Finally, in the validation process during model aggregation, we adopt a random single-layer validation method instead of full-layer model validation, accelerating the validation process of the global model. The main contributions of this paper are as follows:

We propose a decentralized asynchronous federated learning framework for edge devices, which defines the training process, the model storage patterns, and the model validation process.

We propose a dynamic growth cache federated learning (FedDgc) method under Non-IID settings, which reduces early-stage redundant gradient exchanges and keeps the utility of the global model.

We adopt a random single-layer validation method, which distributed validation method that reduces computational burdens on individual edge devices and shortens model validation time.

We conduct experiments on real-world benchmark datasets, including comparisons with other algorithms. The results show that the method trains models with better test performance, reducing computation while maintaining convergence speed.

2. Related work

Many researchers have proposed relevant methods to address existing problems in FL framework. For asynchronous FL, Xie et al. [16] proposed an asynchronous federated optimization algorithm. They demonstrated that the proposed approach exhibits linear convergence properties close to global optimality for strongly convex problems and limited non-convex problem sets. Chen et al. [17] aim to alleviate the impact of local model delays on the global model during asynchronous updates. They used dynamic learning rates to compensate for stale models and introduced decay coefficients to balance inter-client correlations. Although papers [16,17] reduce the waiting time associated with parameter servers, which still fall short of fully exploiting idle devices for training and may lead to a decrease in model generalization performance due to non-IID dataset.

For the non-IID dataset, Zhou et al. [22] proposed a time-efficient asynchronous federated learning protocol. They improved the accuracy of the global model by incorporating a caching mechanism and introducing model staleness parameters. Wang et al. [23] designed a novel centralized fusion algorithm for global model updates. This algorithm determines the aggregation weights for each local model to synchronize training times as closely as possible with the proportion of user device datasets and samples. Ma et al. [24] conducted theoretical analyses of the proposed algorithm, exploring the relationship among

convergence bounds, the number of participants in each round, and the degree of non-IID data. Nguyen et al. [25] based on a caching mechanism, provided theoretical insights into convergence behavior in non-convex settings and selected suitable cache sizes based on experimental results. Zhou et al. [26] improved convergence speed by selecting gradients with similar descent directions, thereby reducing the influence of non-IID dataset, and improving prediction accuracy by dynamically adjusting the learning rate. While all the above works use caching mechanisms to increase the number of aggregated local models, they use a fixed cache capacity to receive local models. In contrast, our proposed Dynamic Growth Cache Federated Learning (FedDgc) method differs from the fixed cache methods mentioned above. FedDgc dynamically adjusts cache capacity during the training process, thereby reducing redundant gradient exchanges during the rapid convergence phase in the early stages of training.

For blockchain and FL integration, Li et al. [27] proposed a Blockchain-based Federated Learning with Caching framework. This framework uses blockchain for global model storage and local model update exchange. Al Mallah et al. [28] employed miners to aggregate received local model updates while utilizing behavior proofs to select reliable training devices. Feng et al. [29] addressed latency in FL by controlling block generation rates and dynamically adjusting the number of training iterations to prevent frequent uploads of local models, which could lead to blockchain transaction overload. Liu et al. [30] focused on preventing malicious participants from poisoning the training process. Miners verified uploaded local models through cross-validation, and the resulting global model was uploaded to the blockchain after consensus validation. Paper [27–30] involve uploading local models to the blockchain for storage and employing miners for local model aggregation to generate the global model, which is then uploaded on the blockchain. However, uploading large volumes of data to the blockchain will reduce its operational speed. To alleviate this, paper [31–33] used gradient compression to reduce the data volume of models. In our proposed framework, we utilize the interplanetary file system (IPFS) [34] to store models and record storage address content identifier (CID) on the blockchain, thereby reducing the amount of data uploaded on the blockchain to enhance efficiency.

3. Proposed framework

3.1. Federated learning

For ease of expression, some key notations in this paper are listed in Table 1. This framework considers that there are N edge devices participating in FL training, and edge device has a local dataset k . Assume that different edge devices have different local datasets, i.e. $D_k \neq D_j, \forall k \neq j$, x_k represents a single-sample sampling from D_k . The loss function for edge device k is defined as follows:

$$F_k(w_k) = \frac{1}{|D_k|} \sum_{x_k \in D_k} f(w_k; x_k) \quad (1)$$

where w_k is local model parameters. We also set the loss function to $f(w_k; x_k) = l(w_k; x_k)$. We define $n_k = |D_k|$ as the number of samples on edge device k and denote $n = \sum_{k=1}^N n_k$ as the total number of samples across all edge devices participating in the training. The global model loss function over all datasets owned by edge devices can be defined as follows:

$$F(w) = \sum_{k=1}^N \frac{n_k}{n} F_k(w_k) \quad (2)$$

the optimization objective of FL is to obtain the global model parameters w_* that minimize the resulting $F(w)$, i.e. $w_* = \min_{w \in \mathbb{R}^d} F(w)$.

Table 1
Experimental setup.

Symbol	Semantics
k	Edge device participating in FL
D_k	The local dataset on edge device k
x_k	The batch of D_k
n_k	The number of samples on edge device k
$F(w)$	The global loss function
$F_k(w_k)$	The global loss function
$F(w_*)$	The optimal value of function $F(w)$
w^r	The global model in round r
w_k^{la}	The local model derived by local updating w^r
$\bar{w}^r$	The aggregated model in round r
t_k	The timestamp of the model w^r
ρ	The regularization weight
μ^r	The cache capacity in round r
τ_k	The delay of edge device k
τ^m	The maximum update delay
$S_k()$	The staleness function
$\bar{\tau}$	The average staleness
α^r	The mixing hyperparameter in round r
t	The timestamp of the global round

3.2. Framework components

The involvement of numerous edge devices in FL increases the importance of effectively addressing trust issues during the training process. Blockchain provides a reliable guarantee for FL involving edge devices as a decentralized technology. It effectively alleviates security concerns and privacy challenges associated with centralized servers. Using blockchain technology to build trusted FL clusters shifts large-scale centralized FL to small-scale distributed learning, providing a solution for more organizations or individuals.

In traditional FL, the parameter servers select all or a subset of training devices to participate in model training during global iterations. Selecting a large number of training devices will increase the waiting time for parameter servers. Conversely, selecting too few training devices would waste the resources of those not selected. To address this issue, we propose Dynamic Growth Cache Asynchronous Federated Learning (more details are described in 3.3). Edge devices can proactively participate in FL training tasks during idle times, reducing waiting time for model updates. In addition, the cache size is dynamically adjusted based on the number of training devices and training progress.

Our proposed decentralized asynchronous federated learning framework for edge devices is shown in Fig. 1. The overall architecture includes an edge device layer, an IPFS layer, an aggregation layer and a blockchain layer. The layers interact with each other through model parameters to ensure secure sharing and updating of models.

Edge Devices Layer: First, the external device is registered as a training device through a smart contract deployed on the blockchain (step a). Next, the devices collect local datasets suitable for Federation Learning (FL) tasks (step b). When the devices become available, they download the global model stored on IPFS and train the local model using the local dataset (steps c and e). Finally, the devices upload the local model parameters to IPFS when training is complete (step d).

IPFS Layer: Both global and local models are stored in IPFS for devices to download and upload. When a device uploads models to IPFS, IPFS generates a unique CID for each model. To accelerate model access and retrieval, a model caching mechanism is designed. This ensures that the device can quickly obtain the required models (step g). Meanwhile, the model CIDs uploaded by the device are encrypted and stored securely in the blockchain. It guarantees data privacy and model traceability.

Blockchain Layer: At the blockchain layer, the security and validity of the model parameters are guaranteed. First, the consensus group encrypts the CIDs uploaded by the devices with RSA using the public

key to prevent unauthorized access to the local models by other devices (step f). After that, the consensus group node extracts the encrypted CID from the blockchain and downloads the corresponding local model (step h). Finally, local model validation is performed. If local model validation passes, all validated local models will be aggregated by the aggregation node to generate a new global model.

Aggregation Layer: First, the aggregation node extracts the encrypted local model CID from the blockchain and downloads the corresponding model parameters. The verified local model is passed to the aggregation node (step h). The validation node verifies the generated global model using our proposed random single-layer validation mechanism (more details are being described in 3.4) and decides the correctness of the model through a voting mechanism (step i). When the voting pass rate reaches the requirement, the new global model will be obtained (step j). Next, the system uploads the new global model and timestamp to IPFS (step k). Immediately after that, the global model will be used by IPFS to generate new CIDs (step l). Finally, the updated global model and its corresponding CID will be distributed to all participating edge devices to ensure that each device can use the latest global model for the next round of training.

3.3. FedDgc

The proposed dynamic growth cache asynchronous federated learning method involves a consensus group and multiple edge devices. The consensus group aggregates models stored in its cache when the model cache meets the update threshold. To alleviate the staleness of the local model on the global model during training, a staleness function is used for weighted models. Furthermore, the dataset volume weight is introduced for balance as it affects the training time. To alleviate the impact of the Non-IID dataset, a decay term is added to the model optimization objective.

Training initialization: At the start of FL training, consensus group members upload the initial global model to the IPFS and store the CID on the blockchain. At the same time, the model cache L is cleared, awaiting the participation of idle edge devices in the FL training.

Training Participation: Differing from the methods of designating specific edge devices for participation in training, FedDgc adopts a proactive pattern for edge devices. This pattern effectively utilizes the resources of idle edge devices for training, reducing the waiting time during global model updates. In the round r of global model iteration, when an idle edge device k becomes a training device through the blockchain, it downloads the global model w^r from IPFS using the global model CID stored on the blockchain. The global model w^r is then used as a local model w_k for training and t_k is the timestamp of the model w_k .

Local Training: After obtaining the local model w_k , the edge device k proceeds to train the model using its local dataset. In FL, the common training is to use the loss function $f(w_k; x_k) = l(w_k; x_k)$ directly as the optimization objective. After the neural network completes a mini-batch of forward passes, it computes the loss function results. And the loss function is differentiated by back propagation to update the weight parameters of the neural network. However, participating edge devices hold non-IID datasets. This may cause the convergence of the global model to slow down or even cause it to diverge when the local model is aggregated with the global model. Therefore, the edge device k introduces a decay term when optimizing the objective in local training instead of simply using the loss function as the only optimization objective. The training objective is defined as follows:

$$g(w_k; x_k) = f(w_k; x_k) + \frac{\rho}{2} \|w_k - w^r\|^2 \quad (3)$$

where ρ is the regularization weight parameter, which is used to alleviate the bias between the local model and the global model. This loss function alleviates the adverse effects arising from the heterogeneity of local dataset on edge devices. Subsequently, a random mini-batch of data x_k is selected from the dataset D_k owned by the edge device k as

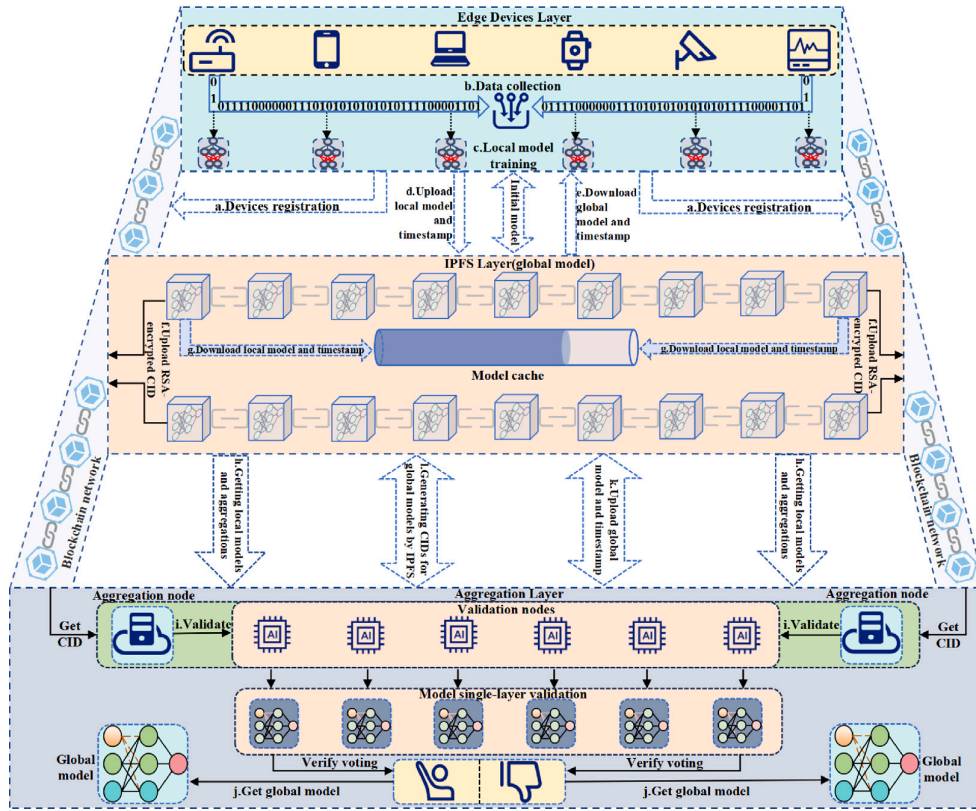


Fig. 1. A decentralized asynchronous federated learning framework for edge devices.

the input for the current local training iteration. The update of the local model is defined as follows:

$$w_k^{t_k} = w_k - \eta \nabla g(w_k; x_k) \quad (4)$$

where η is the learning rate. After completing the local model training, the edge device k uploads the trained local model $w_k^{t_k}$ to IPFS, and records the CID and timestamp t_k on the blockchain, finalizing the model update for the round t . Subsequently, this edge device may exit the FL due to factors such as network conditions or energy constraints, or it may choose to download the latest global model w^{t+i} , $i \in \mathbb{N}$ for local training.

Dynamic Caching Mechanism: In FL, the parameter server aggregates the received local models to generate the global model. However, due to the Non-IID datasets owned by participating edge devices, the local models generated by different devices exhibit significant differences [35]. The impact of IID and Non-IID datasets on the local models and the global model is shown in Fig. 2. During the global iteration at the round t , w_1^t, w_k^t represent the convergence directions of the local models from edge devices 1 and k , respectively. The w_f^t represents the convergence direction when using the FedAvg method to generate the global model. The w_c^t represents the convergence direction when using the SGD method to generate the global model.

We use arrows to represent the convergence directions of models. When the dataset is IID, w_1^t and w_k^t exhibit small deviations in the initial stage. After r rounds of global iterations, w_f^t and w_c^t still maintain relatively close convergence directions. When the dataset is Non-IID, due to the influence of data distribution on the model's convergence direction, the deviations between w_1^t and w_k^t gradually widen and accumulate rapidly. This situation leads to an increased gap between w_f^t and w_c^t , ultimately reduces the model of convergence speed or even causes model divergence.

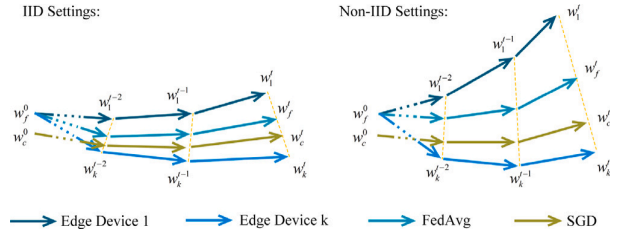


Fig. 2. Illustration of the weight divergence for federated learning with IID and Non-IID data.

In asynchronous FL research, both methods in [16,17] update the global model immediately when the parameter server receives a local model update from any participating training party. This model can speed up the update of the global model in federated learning (FL). Nevertheless, when the dataset on the edge device is a non-IID dataset, changes in the direction of local model convergence may lead to fluctuations and divergence in the global model. To alleviate the impact of local models on the global model, similar to synchronous FL methods, FedDgc generates the global model using a cache mechanism to reduce the influence of non-IID dataset on global model convergence.

In each round of global iteration, the number of aggregated local models also affects the global model. Too few aggregated local models are not conducive to the convergence of the global model, while too many aggregated local models increase the waiting time for the parameter server. Although both methods in [10,11] have mathematically analyzed the cache size and determined the optimal cache size, which is 10% of the number of participating edge devices, with the addition and withdrawal of edge devices during the training process, it is necessary

to dynamically adjust cache size. This ensures the convergence of the global model while reducing the waiting time for global model updating.

Furthermore, global models converge rapidly in the early stages of FL training, which results in an insensitive number of aggregated local models. In other words, using a smaller update threshold in the early stages of training does not significantly impact the convergence speed of the global model. Additionally, the number of edge devices participating in FL training for real-world situations increases gradually over time, rather than having a large number of edge devices participate at the beginning of training. Therefore, dynamically increasing the cache quantity during training can improve training efficiency while ensuring accuracy. In the round r of global model iteration, the cache quantity μ^r is defined as follows:

$$\mu^r = \min(t/v, N^r \lambda) \quad (5)$$

where λ is the optimal cache ratio and v is the dynamic growth interval. The consensus group first determines the number of edge devices participating in training, denoted as N^r . Based on the number of participating edge devices, the cache size is calculated.

Model Staleness: Unlike synchronous FL, asynchronous FL receives local models generated from different global round iterations. These models can slow down the convergence of the global model or even lead to model divergence [36]. Specifically, it is assumed that the device k is updated with the local model on round r given by:

$$w_k^{r+1} = w_k^r - \eta \nabla g(w_k^r; x_k) \quad (6)$$

where w_k^r is the local model trained by device k in round r ; $\nabla g(w_k^r; x_k)$ is the gradient of the training objective function. However, there is a time delay τ_k when device k uploads a local model at round r , i.e., the difference between the timestamp t_k of device k local model at the time of upload and the current global round timestamp t :

$$\tau_k = t - t_k \quad (7)$$

As a result, the update formula for the global model becomes:

$$w^{r+1} = \sum_{k=1}^N p_k (w^{t-\tau_k} - \eta \nabla g(w^{t-\tau_k}; x_k)) \quad (8)$$

where $w^{t-\tau_k}$ represents the global model state at time $t - \tau_k$, considering the delay τ_k in the update from device k .

The update of the global model is approximated based on the Taylor expansion as follows:

$$w^{r+1} \approx w^t - \eta \sum_{k=1}^N p_k \nabla g(w^t; x_k) - \sum_{k=1}^N p_k \tau_k \frac{\partial w}{\partial t} \quad (9)$$

therefore, the delay term $\sum_{k=1}^N p_k \tau_k \frac{\partial w}{\partial t}$ affects the updating direction of the global model, making the global model deviate from the correct optimization direction.

To alleviate the impact of stale local models, FedDgc introduces a maximum update delay parameter τ^m and applies staleness weighting to local models. τ^m is defined as the maximum allowable time difference between the local model timestamp uploaded by the edge device and the current global round time. In this, we first collect delay data for different time periods and network environments. The difference between the timestamp when the edge device uploads the model and the global round time is recorded. Based on these data, we calculate the average and maximum delay of the network and set a safety margin (e.g., 20%–30%) for unexpected situations. When the consensus group receives the local model $w_k^{t_k}$ uploaded by the edge device k along with the model timestamp t_k , it calculates the model delay. If the model delay is less than the maximum update delay τ^m , the model is saved to the model cache, and the cache quantity is updated.

In this paper, the staleness function $S_k(\tau_k)$ of the local model $w_k^{t_k}$ uploaded by the edge device k is defined as follows:

$$S_k(\tau_k) = (\tau_k + 1)^{-a}, a > 0 \quad (10)$$

where τ_k denotes the delay of the local model $w_k^{t_k}$ and a is a hyperparameter that controls the rate of weight decay. At $\tau_k = 0$, $S_k(0) = 1$, which indicates that the no-delay model contributes the maximum weight. As τ_k increases, the weight $S_k(\tau_k)$ monotonically decreases and reduces the influence of the delay model on the global model. A larger hyperparameter a indicates that the weights decay faster, strictly limiting the stale model influence. In contrast, the smaller a allows the stale model to participate in the update moderately. This design alleviates the global model dispersion problem caused by the training delay of the stale model.

Model Aggregation: When the received number of local models equals the update threshold, the consensus group takes over the aggregation of these local models instead of the centralized server. In order to balance the impact of model staleness and dataset size on training time, we perform a weighted average of local models from different edge devices. The global average aggregation model $\bar{w}^r$ is obtained through the above which is defined as follows:

$$\bar{w}^r = \frac{\sum_{k \in L} \frac{n_k}{n} S_k(\tau_k) w_k^{t_k}}{\sum_{k \in L} \frac{n_k}{n} S_k(\tau_k)} \quad (11)$$

where L denotes the set of all devices involved in federated learning. n_k represents the number of samples of the device k and $n = \sum_{k \in L} n_k$. $w_k^{t_k}$ is the local model parameter of device k at the moment.

Model Updates: When updating the global model to prevent significant fluctuations in its convergence direction after aggregation, the global model w^{r+1} is defined as follows:

$$w^{r+1} = \alpha \bar{w}^r + (1 - \alpha) w^r \quad (12)$$

where α is a mixing hyperparameter. α can be adjusted to control the effect of the aggregation model $\bar{w}^r$ on the global model. FedDgc adjusts the mixing hyperparameter α based on the average staleness $S(\bar{\tau})$ which is defined as follows:

$$S(\bar{\tau}) = S\left(\frac{1}{\mu} \sum_{k \in L} (t - t_k)\right) \quad (13)$$

in which μ is the total number of devices participating in federated learning. The device k at the current moment t has a model update time of t_k .

The mixing hyperparameter α^r in the r round of global iteration is defined as follows:

$$\alpha^r = a S(\bar{\tau}) \quad (14)$$

where a is a constant indicating the basic level of influence on the mixing hyperparameters. It is usually set to 0.5. And $S(\bar{\tau})$ is a staleness function calculated from the average staleness of the local model τ . Since a only provides a base impact weight, it does not capture the actual difference in model delay (staleness) in different rounds. Instead, the mixing hyperparameter α^r combines a with the average staleness $\bar{\tau}$, which allows the weights to be dynamically adjusted according to the current delay of the participating devices [37].

Then the update of the global model w^{r+1} in the $r+1$ round is defined as follows:

$$w^{r+1} = \alpha^r \bar{w}^r + (1 - \alpha^r) w^r \quad (15)$$

To ensure the correctness of the generated global model, members of the consensus group perform a single-layer global model verification on the generated global model. Once the model is confirmed, the global model w^{r+1} is uploaded to IPFS, and its address CID is stored on the blockchain. Additionally, after the completion of each round of global iteration, the consensus group checks the timestamp list stored in the blockchain. If the local model timestamp of the edge device k exceeds the maximum update delay, i.e. $t - t_k > \tau^m$, the consensus group sends a notification to the edge device k . Edge device k then stops updating the model $w_k^{t_k}$ and decides whether to continue training or exit training based on its available resources. The specific process of FedDgc is outlined in Algorithm 1.

Algorithm 1 FedDgc

Require: Global model rounds, R ; Initial global model, w^0 ; Maximum update delay, τ^m ; Optimal cache ratio, λ ; Timestamp of global rounds, t ; Dynamic growth interval, v ; Mixing hyperparameter, α ;

Ensure: The final global model, w ;

```

1:  $r = 0, L = \emptyset$ ;
2: for  $r < R$  do
3:   Processing at Each Edge Device  $k$ 
4:   if Download  $w^t$  from the IPFS then
5:     Update timestamp  $t_k = t$ 
6:     Update loss function by Eq. (3)
7:     Update local model  $w_k^{t_k}$  by Eq. (4)
8:     Upload timestamp  $t_k$  and  $w_k^{t_k}$  to IPFS
9:     Store CID on the Blockchain
10:  end if
11:  Processing at the Consensus Group
12:  if Blockchain length increases then
13:    Calculate the number of participating edge devices  $N^t$ 
14:    Calculate model cache quantity  $\mu^t$  by Eq. (5)
15:    Download timestamp  $t_k$  and  $w_k^{t_k}$  from IPFS
16:    if  $t - t_k \leq \tau^m$  then
17:      Store timestamp  $t_k$  and  $w_k^{t_k}$  to model cache  $L$ 
18:    end if
19:    if  $|L| = \mu^t$  then
20:      for  $t_k$  and  $w_k^{t_k}$  in  $L$  do
21:        Calculate model delay  $\tau_k = t - t_k$ 
22:        Calculate staleness  $S_k(\tau_k)$  by Eq. (6)
23:      end for
24:      Aggregate global model by Eq. (7)
25:      Calculate average staleness  $S(\bar{\tau})$  by Eq. (9)
26:      Calculate mixing hyperparameter  $\alpha^r$  by Eq. (10)
27:      Update global model by Eq. (11)
28:      Upload  $w^{r+1}$  to IPFS
29:      Store CID on the Blockchain
30:       $L = \emptyset$ 
31:    end if
32:  end if
33:   $r = r + 1$ 
34: end for
35: return The final global model  $w$ ;

```

3.4. Model random single-layer validation

To address the issue of untrusted third-party parameter servers, we proposed the use of a consensus group to replace the parameter server in aggregating local models. Specifically, when the edge device k stores the CID of a local model uploaded to IPFS in the blockchain through a smart contract, the blockchain's length increases due to the generated transaction. The consensus group monitors changes in the blockchain's length to determine if local models have been uploaded. Upon receiving a local model $w_k^{t_k}$ from the edge device k and verifying its timestamp t_k , the consensus group stores this model in the cache. Model aggregation is performed when the cache size equals the update threshold.

In the existing block generation methods, for example PoW, block-producing nodes continuously adjust the nonce value to calculate a hash that meets the required conditions through SHA256 computation. This process demands significant computational power for nonce value calculation to compete for block generation rights. However, this method is not conducive to the participation of edge nodes and is unsuitable for FL. In FL, it is essential to ensure the correctness of the global model before block generation. With the evolution of neural networks, especially the emergence of deep neural networks, the number of model parameters has grown significantly. Taking the classic

ResNet [38] as an example, it comprises approximately 11 million parameters. Aggregating and verifying all parameters of the global model would consume a substantial amount of time. Therefore, we proposed the use of random single-layer global model verification as a method to validate the generated global model.

To prevent malicious behavior within the consensus group, we randomly select one member as the aggregation node in each round of global iteration. It is responsible for model aggregation and the remaining members act as validation nodes. This approach reduces the risk of manipulation by specific nodes and improves efficiency through single-layer global model validation. Single-layer validation randomly selects one layer of the neural network from which only the parameters of the layer are aggregated. Specifically, the validation node computes the parameters of the aggregated single-layer model and compares them one by one with the parameters of the corresponding layer in the full-layer model provided by the aggregation node. If the parameters are within the preset 4% tolerance error, the aggregation result is considered correct. In addition, a threshold of 2/3 voting ratio is set and the aggregation result is accepted only if this ratio is reached. If the vote of a validation node is significantly inconsistent with other nodes, the blockchain anomaly detection mechanism will be triggered to assess the reliability of the node. At the same time, the historical performance of the validation node is also used as a basis for voting. In case a node repeatedly makes wrong judgments, its voting weight will be adjusted accordingly. The mechanism ensures that only models that are approved by a majority of validation nodes are accepted. If the aggregation result is rejected, the aggregation node is removed, and a new member is selected to join the consensus group.

Due to the following characteristics of IPFS: each different content generates a different storage address (CID), while the same content added to IPFS from different nodes will have the same storage address (CID), consistency between the model and the storage address can be ensured. After the model verification is completed, three consensus group members are randomly selected to upload the global model to IPFS. If two members obtain the same storage address, that address is uploaded to the blockchain network. This approach prevents malicious tampering with the global model during the upload process, as blockchain nodes only need to maintain the consistency of block information without verifying the correctness of the content.

4. Evaluations

To evaluate the effectiveness of the proposed method and the impact of different hyperparameters on the FL process, we conducted simulation experiments to investigate the convergence of FL tasks on different datasets. The experiments were performed on a computing environment equipped with an AMD Ryzen 5 1600X CPU and an NVIDIA GeForce RTX 3060 GPU. The software stack included Python 3.9, CUDA v11.7, and cuDNN v8.8.0.

4.1. Experimental setup

Datasets: The experiments were conducted using two publicly available datasets, CIFAR-10 [39] and Fashion-MNIST [40]. The CIFAR-10 dataset consists of color images with 10 different classes, comprising 50,000 training images and 10,000 test images. Fashion-MNIST contains grayscale images categorized into 10 classes, with a total of 60,000 training images and 10,000 test images.

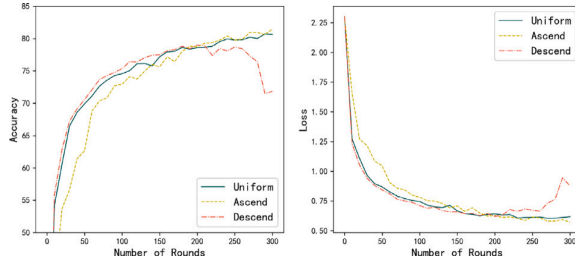
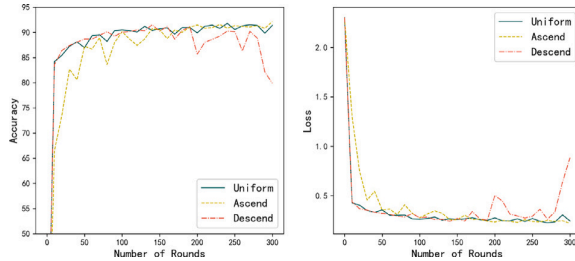
Implementation of Non-IID: The CIFAR-10 and Fashion-MNIST datasets were partitioned into 100 subsets, with each subset corresponding to the local data held by an edge device. The partitioning of the CIFAR-10 and Fashion-MNIST datasets was performed according to a Dirichlet distribution [41], resulting in each device dataset being Non-IID and of different sizes.

Model: In our experiments, a convolutional neural network (CNN) was used to train FL on the Non-IID datasets. This CNN model consists

Table 2

Experimental setup.

Symbol	Symbol	Setting
Total number of edge devices	N	100
Local epochs	E	1
Global epochs	T	2000
Mini-batch size	B	10
Learning rate	η	0.01
Cache ratio	λ	0.1

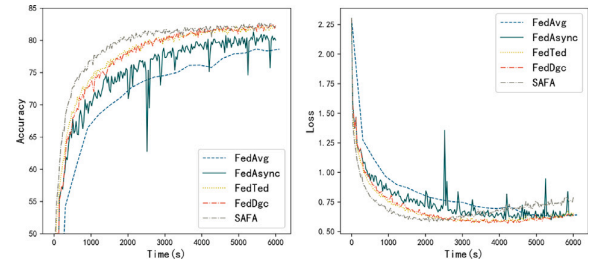
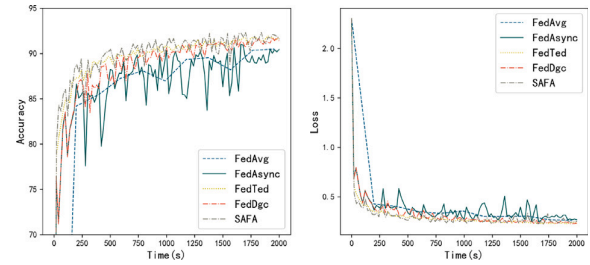
**Fig. 3.** The effect of different aggregate modes on the performance of the FedDgc global model in the CIFAR-10 dataset. Left: Accuracy; Right: Loss.**Fig. 4.** The effect of different aggregate modes on the performance of the FedDgc global model in the Fashion-MNIST dataset. Left: Accuracy; Right: Loss.

of four convolutional layers, each with 3×3 kernels, followed by two fully connected layers and a softmax output layer. This lightweight convolutional neural network is suitable for computation on edge devices and is identical to the experimental model used in [16]. The detailed experimental settings are shown in Table 2.

4.2. Experimental analysis

Effect of number of aggregates: In FL process, selecting as many training devices as possible is advantageous for enhancing the ultimate results of global model, encompassing training loss and accuracy. However, increasing the number of devices involved in training faces many challenges due to the limitations of computational power, network conditions, and data availability of edge devices in actual training. To verify the effect of the number of aggregated local models on the global model during global iterations, three aggregation patterns were trained: Ascend, Uniform, and Descend. Here, Ascend indicates a gradual increase in the number of aggregated local models from 1 to 20, Uniform keeps the number of aggregated local models constant at 10, and Descend indicates a gradual decrease in the number of aggregated local models from 20 to 1. All three patterns resulted in an average of 10 aggregated local models per global iteration, as shown in Figs. 3 and 4.

It is clear that the difference in the number of models leads to different performance of the global model, despite the same average number of aggregations for the global iterations. Overall, the Ascend achieves better training performance than Uniform and Descend. In the early stages of training, both Uniform and Descend converge quickly,

**Fig. 5.** The performance of the FedDgc model compared to different methods on the CIFAR-10 dataset. Left: Accuracy; Right: Loss.**Fig. 6.** The performance of the FedDgc model compared to different methods on the Fashion-MNIST dataset. Left: Accuracy; Right: Loss.

while Ascend, due to its lower number of aggregated local models, exhibits slightly slower convergence. In the later stages of training, the Descend, with fewer aggregated local models, experiences significant model fluctuations due to the impact of the Non-IID dataset, leading to a substantial decrease in model accuracy.

This result indicates that during the initial rapid convergence phase of training, the learning performance of the model exhibits a limited sensitivity to the local model aggregation models. However, the global model is more susceptible to local model aggregation at later stages. Dramatic changes in the direction of local model convergence may lead to a decrease in the accuracy of the global model. Based on these results, FedDgc dynamically increases the cache size during training progression, thereby reducing communication rounds while ensuring the utility of the global model.

Effect of training methods: To verify the effect of FedDgc on the global model, we adopt three typical methods as benchmarks for performance comparison. These include FedAvg [42], FedASync [16], SAFA [43] and FedTed [22], all tested under identical experimental conditions. FedAvg and FedASync are classic methods for synchronous and asynchronous FL, respectively, while FedTed and SAFA utilize a fixed-length cache for asynchronous FL with a cache size of 10. In FedAvg, 10 edge devices are selected for training in each global iteration. All three asynchronous methods experimentally set the maximum update delay $\tau^m = 4$, while FedDgc uses a dynamic growth interval $v = 100$. The experimental results are shown in Figs. 5 and 6.

After 6000s of training on the CIFAR-10 dataset, the accuracy of FedDgc, FedTed, SAFA, FedASync and FedAvg were 82.29%, 81.89%, 81.81%, 80.04%, and 78.61%, respectively. On the Fashion-MNIST dataset, after the 2000s of training, FedDgc, FedTed, SAFA, FedASync and FedAvg achieved the accuracy of 91.43%, 91.82%, 91.10%, 90.43%, and 90.48%, respectively. Across both datasets, it was observed that FedDgc achieves better training performance than FedAvg and FedASync while showing performance close to FedTed and SAFA. In addition, FedDgc and FedTed employ a caching mechanism in asynchronous FL due to the use of non-IID datasets. The global models trained by these two methods exhibit less fluctuation in accuracy and training loss compared to the FedASync method. Contrasting SAFA

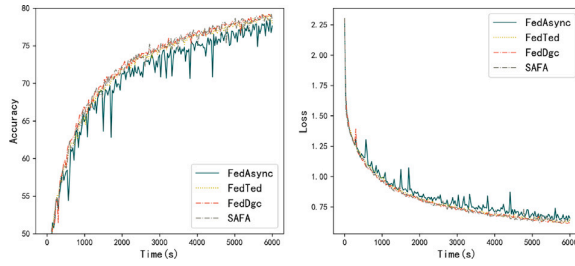


Fig. 7. The performance of the FedDgc model compared to other methods on the CIFAR-10 dataset for $\tau^m = 16$. Left: Accuracy; Right: Loss.

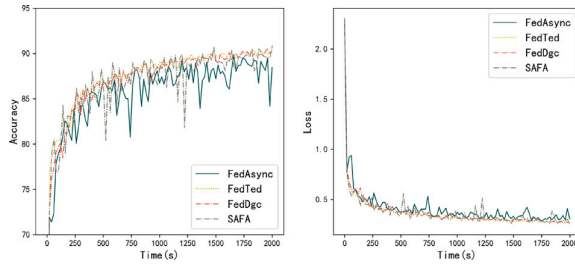


Fig. 8. The performance of the FedDgc model compared to other methods on the Fashion-MNIST dataset for $\tau^m = 16$. Left: Accuracy; Right: Loss.

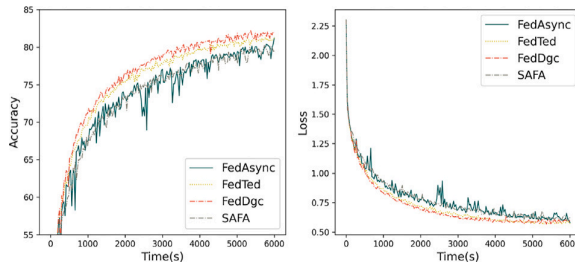


Fig. 9. The performance of the FedDgc model compared to other methods on the CIFAR-10 dataset for $\tau^m = 8$. Left: Accuracy; Right: Loss.

methods that do not use a staleness mechanism results in significant fluctuations in the convergence of the global model.

At time points where the CNN model accuracy reached 80% on the CIFAR-10 dataset and 90% on the Fashion-MNIST dataset, the convergence times for FedDgc, FedTed, SAFA and FedAsync were 2760s, 2585s, 1805s and 4170s for CIFAR-10 and 780 s, 703 s, 463 s and 1380s for Fashion-MNIST, respectively. It can be seen that FedDgc, FedTed and SAFA using model caching converge the global model faster compared to FedAsync. Although FedDgc was slower than FedTed in terms of time, when considering the gradient exchange count [6], FedDgc, FedTed and SAFA had exchanged counts of 4700, 8600 and 6000 for CIFAR-10, with FedDgc reducing the exchange count by 45.34% and 21.67% compared to FedTed and SAFA. For Fashion-MNIST, the exchange counts were 960, 3500 and 2300, with FedDgc reducing the exchange count by 72.57% and 58.26%. FedDgc effectively reduced gradient exchanges while maintaining the utility of the global model.

Effect of maximum update delay: In the asynchronous updating process, the received local models are generated by different rounds of global model training. To verify the effectiveness of the maximum update delay, experiments were conducted by setting the maximum local model update delay $\tau^m = 16$. It is capable of simulating the higher delay situations encountered in real-world applications. The experimental results are shown in Figs. 7 and 8.

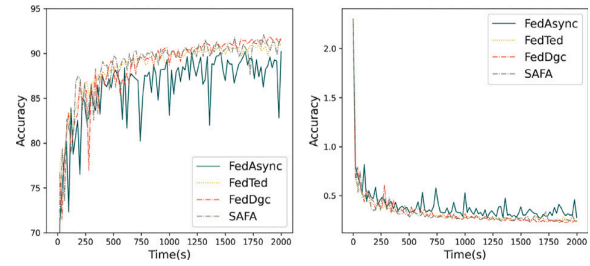


Fig. 10. The performance of the FedDgc model compared to other methods on the Fashion-MNIST dataset for $\tau^m = 8$. Left: Accuracy; Right: Loss.

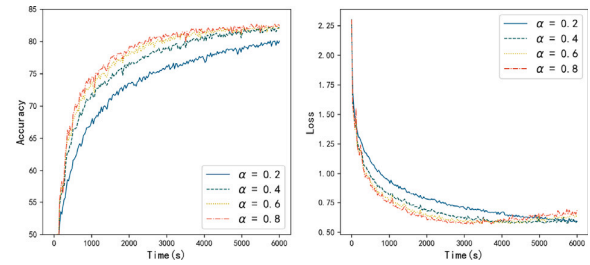


Fig. 11. The performance of FedDgc model on CIFAR-10 dataset for different mixing hyperparameters α . Left: Accuracy; Right: Loss.

In general, the global model accuracy was negatively affected by all four methods as the local model delay increased. After 6000s of training on the CIFAR-10 dataset, the accuracy of FedDgc, FedTed, SAFA and FedAsync were 78.76%, 77.97%, 78.32% and 77.63%, respectively. After 2000 s of training on the Fashion-MNIST dataset, the accuracy of FedDgc, FedTed, SAFA and FedAsync were 90.30%, 90.08%, 90.89% and 88.46%, respectively. The figure shows that FedAsync fluctuates significantly at high delay. This is due to the fact that FedAsync directly relies on asynchronous updates lacking a suitable weight adjustment mechanism. This leads to a decrease in global model accuracy and stability. FedTed and SAFA show some stability on heterogeneous data, but the model performance is lower than the proposed method. This is reflected in the lack of a dedicated delay optimization strategy. In contrast, the proposed method FedDgc adopts a global weighted average update strategy (details in Eq. (10)), which is able to dynamically adjust the weight of the local model in the global aggregation. When the upload delay of a local model is large, FedDgc automatically reduces the weight of that model according to the degree of delay. So that its impact on the global model is reduced, which effectively reduces the interference introduced by the delayed model. Therefore, FedDgc maintains a smaller decrease in accuracy and a smoother increase in loss under high delay conditions. This also indicates robustness to high delay local modeling.

Figs. 9 and 10 shows that the model is updated more frequently at lower delays $\tau^m = 8$. This enables the global model to quickly aggregate local update information, which leads to more efficient convergence and improved accuracy. In addition, FedDgc's adaptive parameter aggregation strategy helps to mitigate model outdated and optimize the utilization of high-frequency updates. This result further demonstrates the effectiveness of FedDgc under different maximum delay conditions.

Effect of mixing hyperparameter: The mixing hyperparameter α controls the extent of global model updates. A larger α means that the aggregated model has a greater influence on model updates, while a smaller α allows local updates to play a more significant role. This can help in identifying and filtering out potential free-riding nodes, as the global model's reliance on local updates becomes more pronounced.

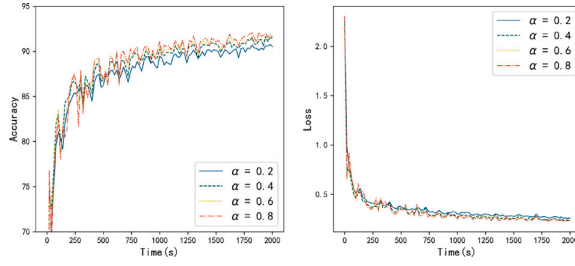


Fig. 12. The performance of FedDgc model on Fashion-MNIST dataset for different mixing hyperparameters α . Left: Accuracy; Right: Loss.

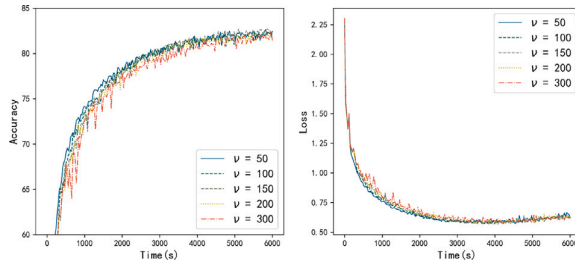


Fig. 13. The performance of the FedDgc model on the CIFAR-10 dataset for different dynamic growth intervals ν . Left: Accuracy; Right: Loss.

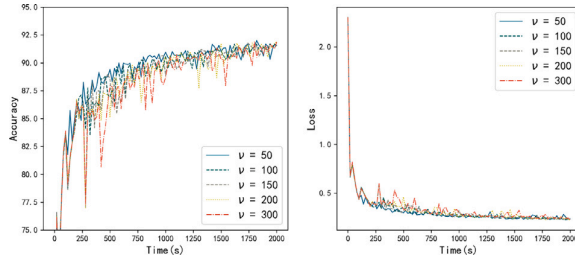


Fig. 14. The performance of the FedDgc model on the Fashion-MNIST dataset for different dynamic growth intervals ν . Left: Accuracy; Right: Loss.

To verify the effectiveness of α on the global model and its role in addressing free-riding behavior, we conducted experiments with different values of $\alpha = 0.2, 0.4, 0.6, 0.8$ for FedDgc. The experimental results are shown in Figs. 11 and 12. After 6000s of training on the CIFAR-10 dataset, FedDgc with $\alpha = 0.2, 0.4, 0.6, 0.8$ achieved the accuracy of 80.04%, 82.11%, 82.29%, and 82.40%, respectively. On the Fashion-MNIST dataset, the global model accuracy is 90.52%, 91.43%, 91.44%, and 91.57%, respectively. The global model accuracy on both datasets increased as α increased, indicating that adjusting α can effectively accelerate model convergence and improve model utility. Importantly, this adjustment may also mitigate the influence of free-riding nodes, as a larger α could mean that only those nodes providing meaningful updates are retained in the aggregation process.

Effect of dynamic growth interval: The dynamic cache growth mechanism in FedDgc enables a decrease in gradient exchanges during the early stages of FL training, which may sacrifice some convergence speed but allows for the identification of contributing nodes. To verify the effectiveness of different dynamic growth intervals on the global model and its implications for free-riding behavior, experiments were conducted with dynamic growth intervals set to $\nu = 50, 100, 200, 300$ and the experimental results are shown in Figs. 13 and 14.

Overall, an increase in the dynamic growth interval leads to a decrease in the convergence speed of the model. This is due to the reduction in the number of local model aggregations, as well as the

Table 3

Time of model validation methods.

Number of aggregation	Single-layer validation	Full-layer validation
1	0.0118	0.1706
2	0.0112	0.1884
3	0.0100	0.1853
4	0.0112	0.1875
5	0.0105	0.1875
6	0.0113	0.1868
7	0.0116	0.1919
8	0.0105	0.1874
9	0.0116	0.1912
10	0.0106	0.1906

Table 4

IPFS upload and download delay for five edge devices.

Edge device	IPFS upload delay (s)	IPFS download delay (s)
Device 1	0.1292	0.1702
Device 2	0.1055	0.1885
Device 3	0.1184	0.1677
Device 4	0.1073	0.1572
Device 5	0.1114	0.1497

influence of the Non-IID dataset, which causes significant fluctuations in the accuracy and training loss of the global model. After 6000s of training on the CIFAR-10 dataset, the accuracy for $\nu = 50, 100, 200, 300$ is 82.43%, 82.29%, 82.27%, and 81.44%, respectively. With the increase in the dynamic growth interval, the global model accuracy gradually decreases, and the convergence speed slows down. This indicates that longer time intervals may reduce the opportunity to aggregate valuable localized updates, potentially benefiting free-riding nodes. In the case of the Fashion-MNIST dataset, which converges relatively quickly within the 2000s of training, the accuracy is 91.58%, 91.43%, 91.65%, and 91.68%, respectively. The model accuracy does not exhibit regular changes due to the increase in the dynamic growth interval, but the overall trend indicates a slowdown in the convergence speed. This reduction in the number of gradient exchanges could enable the identification and exclusion of less effective nodes, thereby mitigating the influence of free-riding behavior. These experimental results demonstrate that careful selection of the dynamic growth interval is crucial in balancing convergence speed and model utility while addressing potential free-riding issues.

Time of model validation methods: To compare the verification speed of the single-layer model validation method with the full-layer model validation, we conducted experiments on the CNN model used in the FedDgc. The model aggregation quantity ranged from 1 to 10, and the average time was calculated from 5 runs. The experimental results are presented in Table 3.

Table 3 shows that the validation times of the single-layer and full-layer model validation methods are approximately 0.01 s and 0.18 s respectively. As the number of aggregated models increases, the model aggregation time slightly increases, but this change is considerably limited. The single-layer model validation method introduces a transformation from sequential to parallel validation, thereby reducing the resource consumption of individual validation devices. However, as the depth of the CNN increases, consensus group members using the single-layer method may face limitations in validating all model parameters. To ensure the correctness of the global model, it becomes necessary to either augment the number of consensus group members or increase the number of layers validated by each consensus group member. Consequently, adopting single-layer model validation during the verification process can effectively reduce validation time and enhance the efficiency of global model validation.

Table 4 shows the average delays when modeling IPFS uploads and downloads for five devices selected from multiple edge devices. Overall,

the upload delay for these devices ranges from 0.1055 s to 0.1292 s. The download delays ranged from 0.1497 s to 0.1885 s. Although the delays differ from one device to another, the overall delay is kept within a small range. It shows the feasibility of adding IPFS to the proposed method FedDgc.

5. Conclusion

In this paper, we propose a decentralized asynchronous federated learning framework for edge devices. We leverage smart contracts deployed on a blockchain to organize and involve edge devices in FL training. To enhance the efficiency of FL, We propose a dynamic growth cache federated learning (FedDgc) method under Non-IID settings, dynamically adjusting cache size throughout the training process. This method alleviates the influence of the Non-IID dataset on the global model while reducing early-stage redundant gradient exchanges, and keeping the utility of the global model. Additionally, for global model validation after consensus group aggregation, we adopt a random single-layer validation method. This distributed validation method reduces computational burdens on individual edge devices and shortens model validation time. Experimental results demonstrate that we train models with better test performance, reducing computation while maintaining convergence speed.

CRedit authorship contribution statement

Bin Wang: Writing – original draft, Visualization, Validation, Software, Methodology, Formal analysis, Data curation, Conceptualization. **Zhao Tian:** Supervision, Project administration. **Jie Ma:** Visualization, Software, Methodology, Data curation. **Wenju Zhang:** Validation, Software, Data curation. **Wei She:** Supervision, Project administration. **Wei Liu:** Writing – review & editing, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by Henan Key Laboratory of Network Cryptography Technology, China (Grant NO. LNCT2022-A04) and the Key Scientific Research Project of Colleges and Universities in Henan, China Province NO.24A520045.

Data availability

Data will be made available on request.

References

- [1] Elarbi Badidi, Karima Moumane, Firdaus El Ghazi, Opportunities, applications, and challenges of edge-AI enabled video analytics in smart cities: A systematic review, *IEEE Access* 11 (2023) 80543–80572.
- [2] ChingHu Lu, XiaoZong Lin, Toward direct edge-to-edge transfer learning for IoT-enabled edge cameras, *IEEE Internet Things J.* 8 (6) (2021) 4931–4943.
- [3] Praveen Joshi, Mohammed Hasanuzzaman, Chandra Thapa, Haithem Afi, Ted Scully, Enabling all in-edge deep learning: A literature review, *IEEE Access* 11 (2023) 3431–3460.
- [4] Chen Zhang, Yu Xie, Hang Bai, Bin Yu, Weihong Li, Yuan Gao, A survey on federated learning, *Knowl.-Based Syst.* 216 (2021) 106775.
- [5] Weidong Du, Min Li, Liqiang Wu, Yiliang Han, Tanping Zhou, Xiaoyuan Yang, A efficient and robust privacy-preserving framework for cross-device federated learning, *Complex Intell. Syst.* (2023) 1–15.
- [6] Shuzhen Chen, Yangyang Wang, Dongxiao Yu, Ju Ren, Congan Xu, Yanwei Zheng, Privacy-enhanced decentralized federated learning at dynamic edge, *IEEE Trans. Comput.* 72 (8) (2023) 2165–2180.
- [7] Danish Javeed, Muhammad Shahid Saeed, Ijaz Ahmad, Muhammad Adil, Prabhat Kumar, A.K.M. Najmul Islam, Quantum-empowered federated learning and 6G wireless networks for IoT security: Concept, challenges and future directions, *Future Gener. Comput. Syst.* (2024).
- [8] Deepali Kushwaha, Surender Redhu, Christopher G. Brinton, Rajesh M. Hegde, Optimal device selection in federated learning for resource-constrained edge networks, *IEEE Internet Things J.* 10 (12) (2023) 10845–10856.
- [9] Thin Tharaphe Thein, Yoshiaki Shiraishi, Masakatu Morii, Personalized federated learning-based intrusion detection system: Poisoning attack and defense, *Future Gener. Comput. Syst.* 153 (2024) 182–192.
- [10] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, Virginia Smith, Federated optimization in heterogeneous networks, *Proc. Mach. Learn. Syst.* 2 (2020) 429–450.
- [11] Takayuki Nishio, Ryo Yonetani, Client selection for federated learning with heterogeneous resources in mobile edge, in: ICC 2019 - 2019 IEEE International Conference on Communications, ICC, 2019, pp. 1–7.
- [12] Sawsan Abdulrahman, Hanine Tout, Azzam Mourad, Chamseddine Talhi, FedM-CCS: Multicriteria client selection model for optimal IoT federated learning, *IEEE Internet Things J.* 8 (6) (2021) 4723–4735.
- [13] Tiansheng Huang, Weiwei Lin, Wentai Wu, Ligang He, Keqin Li, Albert Y. Zomaya, An efficiency-boosting client selection scheme for federated learning with fairness guarantee, *IEEE Trans. Parallel Distrib. Syst.* 32 (7) (2021) 1552–1564.
- [14] Liran Dong, Yiqing Zhou, Ling Liu, Yanli Qi, Yu Zhang, Age of information based client selection for wireless federated learning with diversified learning capabilities, *IEEE Trans. Mob. Comput.* (2024) 1–12.
- [15] Wei-Bin Kou, Shuai Wang, Guangxu Zhu, Bin Luo, Yingxian Chen, Derrick Wing Kwan Ng, Yik-Chung Wu, Communication resources constrained hierarchical federated learning for end-to-end autonomous driving, in: 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, 2023, pp. 9383–9390.
- [16] Chenhao Xu, Youyang Qu, Yong Xiang, Longxiang Gao, Asynchronous federated learning on heterogeneous devices: A survey, *Comp. Sci. Rev.* 50 (2023) 100595.
- [17] Yujing Chen, Yue Ning, Martin Slawski, Huzefa Rangwala, Asynchronous online federated learning for edge devices with non-IID data, in: 2020 IEEE International Conference on Big Data, Big Data, 2020, pp. 15–24.
- [18] Sonam Tyagi, Ishwari Singh Rajput, Richa Pandey, Federated learning: Applications, security hazards and defense measures, in: 2023 International Conference on Device Intelligence, Computing and Communication Technologies, DICCT, 2023, pp. 477–482.
- [19] S. Revathy, S. Sathya Priya, Security enhanced federated learning approach using blockchain, in: 2022 International Conference on Computer, Power and Communications, ICCPC, 2022, pp. 50–55.
- [20] Qing Cai, Yiqing Zhou, Ling Liu, Yanli Qi, Zhengang Pan, Haoyue Zhang, Collaboration of heterogeneous edge computing paradigms: How to fill the gap between theory and practice, *IEEE Wirel. Commun.* 31 (1) (2024) 110–117.
- [21] Andrew R. Short, Helen C. Leligou, Efsthios Theocharis, Execution of a federated learning process within a smart contract, in: 2021 IEEE International Conference on Consumer Electronics, ICCE, 2021, pp. 1–4.
- [22] Chendi Zhou, Hao Tian, Hong Zhang, Jin Zhang, Mianxiang Dong, Juncheng Jia, TEA-fed: Time-efficient asynchronous federated learning for edge computing, in: CF'21: Proceedings of the 18th ACM International Conference on Computing Frontiers, 2021, pp. 30–37.
- [23] Zhongyu Wang, Zhaoyang Zhang, Yuqing Tian, Qianqian Yang, Hangguan Shan, Wei Wang, Tony Q.S. Quek, Asynchronous federated learning over wireless communication networks, *IEEE Trans. Wireless Commun.* 21 (9) (2022) 6961–6978.
- [24] Qianpiao Ma, Yang Xu, Hongli Xu, Zhida Jiang, Liusheng Huang, He Huang, FedSA: A semi-asynchronous federated learning mechanism in heterogeneous edge computing, *IEEE J. Sel. Areas Commun.* 39 (12) (2021) 3654–3672.
- [25] John Nguyen, Kshitiz Malik, Hongyuan Zhan, Ashkan Yousefpour, Michael G. Rabbat, Mani Malek, Dzmitry Huba, Federated learning with buffered asynchronous aggregation, 2021, *ArXiv*, abs/2106.06639.
- [26] Zihao Zhou, Yanan Li, Xuebin Ren, Shusen Yang, Towards efficient and stable K-asynchronous federated learning with unbounded stale gradients on non-IID data, *IEEE Trans. Parallel Distrib. Syst.* 33 (12) (2022) 3291–3305.
- [27] Yuzheng Li, Chuan Chen, Nan Liu, Huawei Huang, Zibin Zheng, Qiang Yan, A blockchain-based decentralized federated learning framework with committee consensus, *IEEE Netw.* 35 (1) (2021) 234–241.
- [28] Ranwa Al Mallah, David López, Talal Halabi, Blockchain-enabled efficient and secure federated learning in IoT and edge computing networks, in: 2023 International Conference on Computing, Networking and Communications, ICNC, 2023, pp. 511–515.
- [29] Lei Feng, Yiqi Zhao, Shaoyong Guo, Xuesong Qiu, Wenjing Li, Peng Yu, BAFL: A blockchain-based asynchronous federated learning framework, *IEEE Trans. Comput.* 71 (5) (2022) 1092–1103.
- [30] Yinghui Liu, Youyang Qu, Chenhao Xu, Zhicheng Hao, Bruce Gu, Blockchain-enabled asynchronous federated learning in edge computing, *Sensors* 21 (10) (2021) 3335.

- [31] Laizhong Cui, Xiaoxin Su, Zhongxing Ming, Ziteng Chen, Shu Yang, Yipeng Zhou, Wei Xiao, CREAT: Blockchain-assisted compression algorithm of federated learning for content caching in edge computing, *IEEE Internet Things J.* 9 (16) (2022) 14151–14161.
- [32] Yongqiang Lu, Zhaobin Liu, Yiming Huang, Parameters compressed mechanism in federated learning for edge computing, in: 2021 8th IEEE International Conference on Cyber Security and Cloud Computing (CSCloud)/2021 7th IEEE International Conference on Edge Computing and Scalable Cloud, EdgeCom, 2021, pp. 161–166.
- [33] Yuanxiong Guo, Rui Hu, Yanmin Gong, Agent-level differentially private federated learning via compressed model perturbation, in: 2022 IEEE Conference on Communications and Network Security, CNS, 2022, pp. 127–135.
- [34] Eugenia Politou, Efthimios Alepis, Constantinos Patsakis, Fran Casino, Mamoun Alazab, Delegated content erasure in IPFS, *Future Gener. Comput. Syst.* 112 (2020) 956–964.
- [35] Xinyan Li, Huimin Zhao, Wu Deng, IOFL: Intelligent optimization-based federated learning for non-IID data, *IEEE Internet Things J.* (2024).
- [36] Ming Chen, Bingcheng Mao, Tianyi Ma, FedSA: A staleness-aware asynchronous federated learning algorithm with non-IID data, *Future Gener. Comput. Syst.* 120 (2021) 1–12.
- [37] Cong Xie, Sanmi Koyejo, Indranil Gupta, Asynchronous federated optimization, 2019, *arXiv preprint arXiv:1903.03934*.
- [38] Wannu Xu, You-Lei Fu, Dongmei Zhu, ResNet and its application to medical image processing: Research progress and challenges, *Comput. Methods Programs Biomed.* 240 (2023) 107660.
- [39] Jiakang Ho, Kyongseok Park, Dae-Ki Kang, GLNAS: Greedy layer-wise network architecture search for low cost and fast network generation, *Pattern Recognit.* 155 (2024) 110730.
- [40] Han Xiao, Kashif Rasul, Roland Vollgraf, Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms, 2017, *ArXiv*, [abs/1708.07747](https://arxiv.org/abs/1708.07747).
- [41] Dun Zeng, Siqi Liang, Xiangjing Hu, Zenglin Xu, FedLab: A flexible federated learning framework, 2021, *ArXiv*, [abs/2107.11621](https://arxiv.org/abs/2107.11621).
- [42] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, Blaise Agueria y Arcas, Communication-Efficient Learning of Deep Networks from Decentralized Data, in: Aarti Singh, Jerry Zhu (Eds.), *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, in: *Proceedings of Machine Learning Research*, vol. 54, PMLR, 2017, pp. 1273–1282.
- [43] Wentai Wu, Ligang He, Weiwei Lin, Rui Mao, Carsten Maple, Stephen Jarvis, SAFA: A semi-asynchronous protocol for fast federated learning with low overhead, *IEEE Trans. Comput.* 70 (5) (2021) 655–668.



Bin Wang received the M.S. degree from the School of Instrumentation Science and Optoelectronic Engineering, Hefei University of Technology, Hefei, China, in 2022. He is currently a Ph.D. student at the School of Computer and Artificial Intelligence, Zhengzhou University, China. His research interests include federated learning, privacy computing, blockchain, deep learning and machine learning.



Zhao Tian received the B.S. degree in information and computing science from Huazhong Agricultural University, in 2008, the M.S. degree in computer software and theory from Zhengzhou University, in 2011, and the Ph.D. degree in safety technology and engineering from Beijing Jiaotong University, in 2016. He is currently a Lecturer with the School of Cyber Science and Engineering, Zhengzhou University. His research interests include blockchain, information security, artificial intelligence, and intelligent transportation.



Jie Ma received the B.E. degree from the School of Computer Science and Technology, Zhengzhou University, China, in 2020. He is currently a M.E. student at the School of Cyber Science and Engineering, Zhengzhou University, China. His research interests include federated learning, privacy computing and blockchain.



Wenju Zhang School of Cyber Science and Engineering, Zhengzhou University, Zhengzhou 450000, China received the B.S. degree in School of Software, Henan University, in 2021. He is currently a master student with the School of Cyber Science and Engineering, Zhengzhou University, China. His research interests include computer technology, blockchain and knowledge distillation.



Wei She received the B.S. degree in control engineering from the Air Defence Academy of PLA, in 2000, the M.S. degree in software engineering from Hunan University, in 2008, and the Ph.D. degree in computer software and theory from Zhengzhou University, China, in 2013, where he is currently a Professor with the School of Cyber Science and Engineering. His research interests include blockchain technology, machine learning and intelligent systems, information security, the energy Internet, and the Internet healthcare.



Wei Liu received the B.S. and M.S. degrees in information engineering from Zhengzhou University, China, in 2003 and 2008, respectively, and the Ph.D. degree from Tohoku University, Japan, in 2013. He is currently an Associate Professor with the School of Cyber Science and Engineering, Zhengzhou University. His research interests include blockchain, artificial intelligence, information security, wireless mesh networks, and the Internet healthcare.